SQL PORTFOLIO PROJECT

Labor Analytics
Shift Detail Summary

Oracle Fusion | Maintenance | Labor Tracking | GROUPING SETS | LISTAGG

Project Summary

This query produces a hierarchical labor hours summary broken down by plant, shift, and individual resource (technician/operator). It joins work orders to operation resource instances and labor instances, extracts shift identifiers from labor codes via REGEXP, and delivers roll-up totals via GROUPING SETS -- enabling dashboards to show per-technician hours and plant-wide shift summaries in a single query pass.

Thomas Foster

Data Analyst | Oracle Fusion ERP | SQL

github.com/thomasfoster | thomasfoster79@yahoo.com

1 PROJECT OVERVIEW

Platform:	Oracle Fusion ERP -- Maintenance (WIE) & Costing modules
Query Type:	Three-layer CTE + GROUPING SETS aggregation
Grain:	Plant / Shift / Resource (with subtotals at each level)
Key Outputs:	Hours charged, shift ID, on-time flag, planned vs. actual duration
Parameters:	:PLANT_CODE, :P_START_DATE, :P_END_DATE, :SHIFT
Audience:	Plant Managers, Maintenance Supervisors, HR/Payroll

2 BUSINESS PROBLEM

Maintenance operations span multiple plants and shifts. Supervisors needed a report to answer: 'How many hours did each technician charge per shift this week, and how do plant totals roll up?' Without this, shift leads pulled time from separate systems and reconciled manually -- prone to double-counting and shift misclassification.

- Payroll reconciliation required hours-charged by individual labor instance per shift.
- Operations needed on-time work-order completion flags (vs. planned start + 12 hours).
- Plant managers required four roll-up levels: per-technician, per-shift, per-plant, grand total.
- The SHIFT dimension was encoded inside the labor instance code string and had to be parsed with REGEXP.
- Hours stored as minutes for some resources required UOM-aware conversion to hours.

3 TECHNICAL EXPLANATION

- MAIN CTE -- pulls work-order header data with UTC-to-Central timezone conversion. Uses NEXT_DAY() + INTERVAL '12' HOUR to compute the ON_TIME flag.
- WO_SCOPE CTE -- deduplicated work-order IDs and plant codes to efficiently scope the labor join.
- LABOR_DETAIL CTE -- joins operation resources to resource instances to labor instances. Extracts SHIFT (last alpha char of labor code) and UNIQUE_RESOURCE via REGEXP_SUBSTR. Converts minutes to hours. Uses LISTAGG to concatenate labor entries per work order.
- RESOURCE_HOURS CTE -- aggregates total hours per plant/shift/resource.
- Final SELECT -- GROUPING SETS emit four subtotal levels in one query pass. GROUPING() flags identify each level for reporting tools.

Key Techniques: GROUPING SETS, REGEXP_SUBSTR, LISTAGG, FROM_TZ, NEXT_DAY, NVL, COALESCE

4 MASKED SQL -- KEY EXCERPTS

```
-- Timezone conversion + On-Time flag (inside MAIN CTE)
TO_CHAR(CAST(FROM_TZ(CAST(WO.ACTUAL_COMPLETION_DATE AS TIMESTAMP),'UTC')
  AT TIME ZONE 'America/Chicago' AS TIMESTAMP),
  'MM/DD/YYYY HH:MI AM') AS ACTUAL_COMPLETION_DATE,
CASE
  WHEN WO.ACTUAL_COMPLETION_DATE IS NULL THEN 'NO'
  WHEN FROM_TZ(CAST(WO.ACTUAL_COMPLETION_DATE AS TIMESTAMP),'UTC')
         AT TIME ZONE 'America/Chicago'
       <= NEXT_DAY(
            TRUNC(FROM_TZ(CAST(WO.PLANNED_START_DATE AS TIMESTAMP),'UTC')
                   AT TIME ZONE 'America/Chicago'), 1)
          + INTERVAL '12' HOUR
  THEN 'YES'
  ELSE 'NO'
END AS ON_TIME,

-- LABOR_DETAIL CTE: parse shift from labor instance code via REGEXP
TRIM(REGEXP_SUBSTR(WL.LABOR_INSTANCE_NAME, '^[^(]+'))   AS UNIQUE_RESOURCE,
CASE
  WHEN REGEXP_COUNT(WL.LABOR_INSTANCE_CODE,'-') >= 2 THEN
    CASE WHEN REGEXP_LIKE(
               SUBSTR(WL.LABOR_INSTANCE_CODE,
                      INSTR(WL.LABOR_INSTANCE_CODE,'-',1,2)+1,1),
               '^[[:alpha:]]$')
         THEN SUBSTR(WL.LABOR_INSTANCE_CODE,
                     INSTR(WL.LABOR_INSTANCE_CODE,'-',1,2)+1,1)
         ELSE NULL END
  ELSE NULL
END AS SHIFT,

-- Final SELECT: GROUPING SETS for four rollup levels
SELECT
  PLANT_CODE, SHIFT, UNIQUE_RESOURCE,
  ROUND(SUM(TOTAL_HOURS_CHARGED),2) AS TOTAL_HOURS_CHARGED,
  GROUPING(PLANT_CODE)      AS G_PLANT,
  GROUPING(SHIFT)           AS G_SHIFT,
  GROUPING(UNIQUE_RESOURCE) AS G_RESOURCE
FROM RESOURCE_HOURS
WHERE (NVL(SHIFT,'Blank') IN (:SHIFT) OR 'All' IN (:SHIFT || 'All'))
GROUP BY GROUPING SETS (
  (PLANT_CODE, SHIFT, UNIQUE_RESOURCE),  -- detail
  (PLANT_CODE, SHIFT),                   -- shift subtotal
  (PLANT_CODE),                          -- plant subtotal
  ()                                     -- grand total
)
ORDER BY PLANT_CODE NULLS LAST, SHIFT NULLS LAST, UNIQUE_RESOURCE NULLS LAST
```

5 EXAMPLE OUTPUT

Plant	Shift	Resource	Hrs Charged	G_Plant	G_Shift	G_Res
[PLANT_A]	A	J. Smith (Electrician)	42.50	0	0	0

[PLANT_A]	A	R. Jones (Mechanic)	38.00	0	0	0
[PLANT_A]	A	-- Shift A Total --	80.50	0	0	1
[PLANT_A]	B	K. Davis (Mechanic)	45.25	0	0	0
[PLANT_A]	--	-- Plant Total --	175.25	0	1	1
-- Grand	--	-- Grand Total --	412.00	1	1	1

** G_Plant/G_Shift/G_Res = 1 indicates a GROUPING SETS subtotal row. Names masked.*